

Alunos: Pedro Henrique Gimenez - 23102766 e Victória Rodrigues Veloso - 23100460

1. Exercício 1

Para o primeiro exercício, foi desenvolvido um programa em Assembly que realiza a soma de uma matriz com outra transposta, contendo o tamanho (MAX) parametrizável.

```
float A[MAX, MAX], B[MAX, MAX]
for (i=0; i< MAX; i++) {
    for (j=0; j< MAX; j++) {
        A[i,j] = A[i,j] + B[j, i];
    }
}
```

Figura 1. Programa em linguagem C para soma de uma matriz com outra transposta

Para que MAX fosse a única variável parametrizável em .data, a estratégia adotada foi a criação de uma função (*cria_matriz*) que gera matrizes alocando-as na memória de forma dinâmica apenas utilizando MAX como referência de tamanho. Para isso utilizamos o comando 9 do syscall que aloca dinamicamente (MAX*MAX*4) bits na memória heap. A implementação pode ser observada na figura 2.

```
.data
    MAX:          .word 4      # tamanho da matriz parametrizável

.text
main:
    lw    $s2, MAX             # salva MAX em $s2

    move   $a0, $s2            # parâmetro da função = max
    jal    cria_matriz
    move   $s0, $v0            # salva matriz A em $s0

..... RESTO DO CÓDIGO .....

cria_matriz:

    move   $t3, $a0            # salva o tamanho da matriz (MAX) em $t3
    # --- calculando espaço necessário para alocar (max * max * 4) salva em $t5 ---
    mul    $t5, $t3, $t3       # calcula max * max
    sll    $t5, $t5, 2         # multiplica pelo tamanho da word (4 bytes)
    # --- aloca espaço dinamicamente na matriz ---
    li     $v0, 9              # syscall que serve para criar um espaço para alocação na
memória heap
    move   $a0, $t5            # envia para o syscall quantos bytes serão alocados
    syscall                     # endereço alocado está salvo em $v0

..... RESTO DA FUNÇÃO .....
```

Figura 2. Trechos do código exemplificando como realizamos a alocação dinâmica

1.1 Resultado obtido

Para o cálculo das soma, duas matrizes de tamanho MAX=4 idênticas foram montadas pela função, os elementos podem ser visualizados na figura 3 e o resultado esperado pode ser visualizado na figura 4 que foi previamente calculado utilizando um script python. É importante observar que, como foi realizada uma alocação dinâmica, para visualizar a matriz precisamos acessar o heap do data segment.

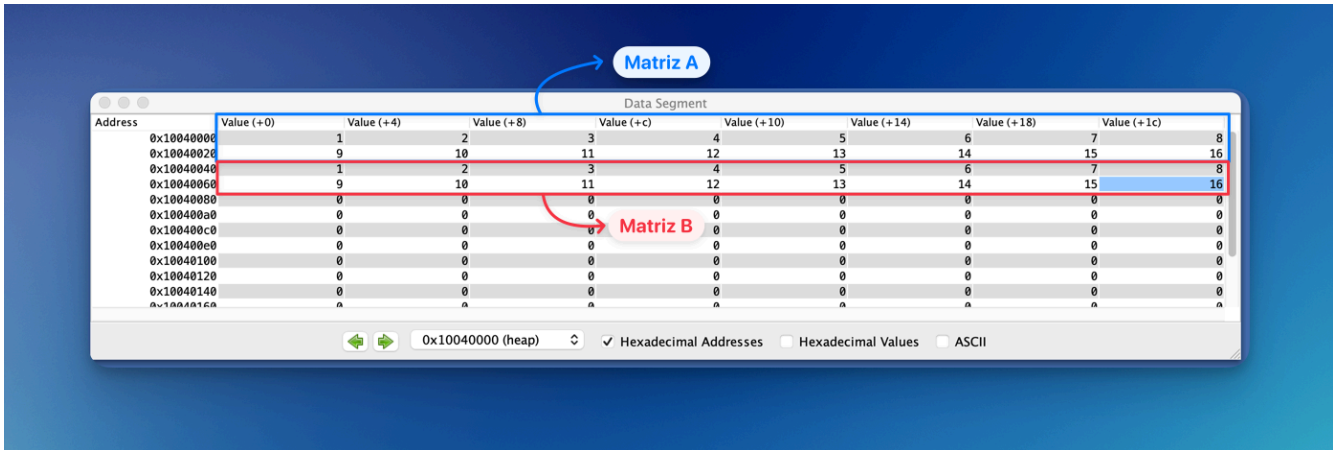


Figura 3. Matrizes A e B alocadas na memória heap



Figura 4. Resultado de como a matriz A deve ficar após a operação, feito em Python

Conforme a figura 5, os resultados obtidos coincidem com os resultados esperados para a operação realizada.

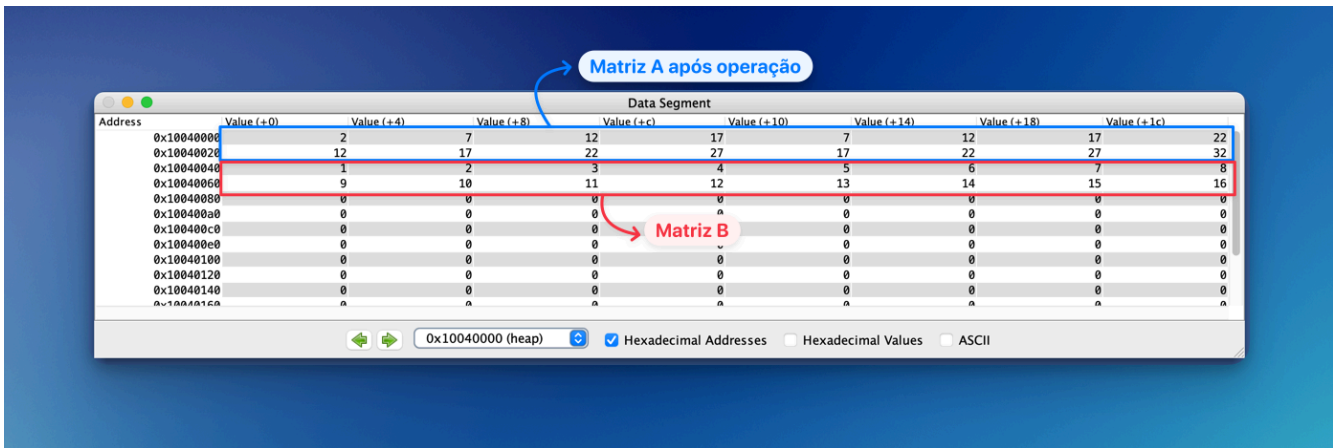


Figura 5. Bloco de memória do Mars após operação do código realizado em assembly

2. Exercício 2

Para o segundo exercício, o mesmo foi proposto, porém com a utilização da técnica cache blocking, conforme figura 6.

```
float A[MAX, MAX], B[MAX, MAX];
for (i=0; i< MAX; i+=block_size) {
    for (j=0; j< MAX; j+=block_size) {
        for (ii=i; ii<i+block_size; ii++) {
            for (jj=j; jj<j+block_size; jj++) {
                A[ii,jj] = A[ii,jj] + B[jj, ii];
            }
        }
    }
    &nbsp; }
}
```

Figura 6. Programa em linguagem C para realizar a soma de uma matriz com outra transposta através da técnica de cache blocking

A estratégia de criação de matrizes dinamicamente também foi utilizada neste exercício. A única parte alterada do código foi a utilização de mais dois for's para ii e jj, como foi descrito pelo pelo problema.

2.1. Resultado obtido

Declaramos 2 matrizes com tamanho MAX=6, como pode ser observado na figura 7 e o resultado esperado pode ser visualizado na figura 8.

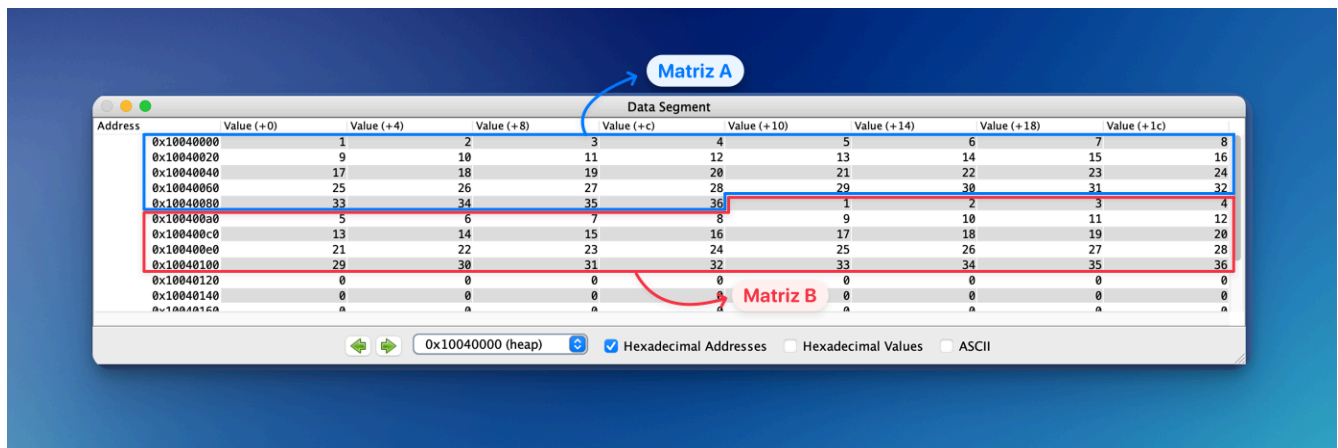


Figura 7. Matrizes A e B alocadas na memória heap

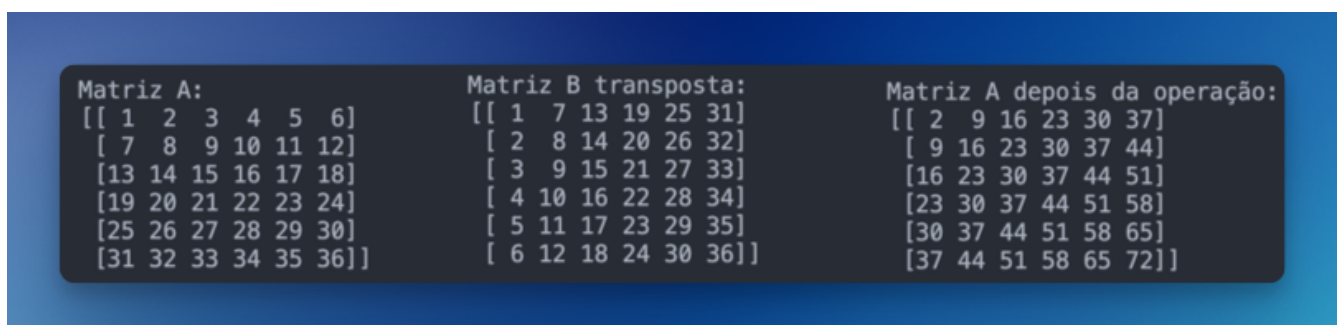


Figura 8. Resultado esperado calculado previamente pelo script em Python

Utilizamos o block_size de tamanho 3. Conforme a figura 9, os resultados obtidos coincidem com os resultados esperados para a operação realizada.

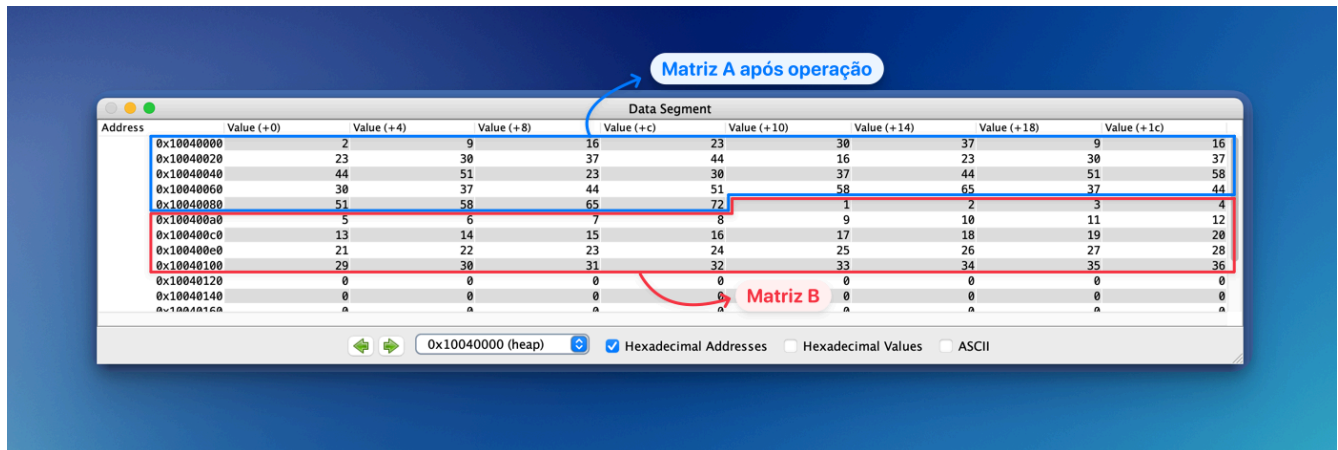


Figura 9. Bloco de memória do Mars após operação do código realizado em assembly

3. Exercício 3

3.1 Simulações para os exercícios 1 e 2

Para as simulações a seguir utilizamos a política de colocação "N-way Set Associative".

Tabela 1. Resultados obtidos para matriz 2x2

Number of blocks	Cache block size	Miss rate Ex. 1	Block_size (ex.2)	Miss rate Ex. 2
8	4	14%	1	14%
16	8	10%	2	9%
32	16	10%	1	9%
64	32	10%	2	9%

Para a matriz 2x2, a técnica de cache blocking não mostrou ter um impacto muito significativo, visto que as taxas de miss rate foram muito próximas para ambas as matrizes devido ao tamanho pequeno da matriz.

Tabela 2. Resultados obtidos para matriz 16x16

Number of blocks	Cache block size	Miss rate Ex. 1	Block_size (ex.2)	Miss rate Ex. 2
8	4	38%	16	38%
16	8	29%	4	17%
32	16	3%	2	3%
64	32	1%	8	1%

Para a matriz 16x16 o mesmo raciocínio da matriz 2x2 pode ser concluído, por não ser uma matriz grande o suficiente para não ‘caber’ na cache, a técnica de cache blocking apresentou resultados semelhantes à matriz do exercício 1, fornecendo taxas de miss rate menores em caches com tamanhos de blocos maiores.

Tabela 3. Resultados obtidos para matriz 200x200 com números de blocos e cache size variando

Number of blocks	Cache block size	Miss rate Ex. 1	Block_size (ex.2)	Miss rate Ex. 2
8	4	38%	20	38%
16	8	29%	100	29%
32	16	24%	50	20%
64	32	22%	40	4%

Para a matriz 200x200, a aplicação de cache blocking mostrou uma redução na taxa de falhas na busca de dados da memória cache. Com um block size pequeno, a taxa de miss rate foi maior, indicando pouco proveito da localidade espacial e que os dados acessados não são suficientemente reutilizados antes de serem substituídos. Por outro lado, block sizes maiores também resultaram em um miss rate elevado, possivelmente devido ao fato de que blocos muito grandes não cabem na cache, levando a substituições sucessivas. Portanto, é necessário balancear o block size para evitar substituições excessivas e garantir que o tamanho do bloco caiba na cache.

3.2 Conclusão

A aplicação de técnicas de cache blocking para otimização de acesso a dados em diferentes tamanhos de matrizes apresentou variações significativas nas taxas de miss rate. Observamos que para matrizes pequenas, como 2x2 e 16x16, a técnica de cache blocking teve um impacto limitado. Devido ao pequeno tamanho das matrizes, elas cabem facilmente no cache, resultando em taxas de miss rate menores com tamanhos de blocos maiores.

Já para matrizes maiores, como 200x200, a aplicação de cache blocking mostrou uma redução na taxa de falhas. Com block sizes pequenos, a taxa de miss rate foi alta devido ao pouco aproveitamento da localidade espacial, resultando em substituições frequentes antes que os dados pudessem ser reutilizados. Com block sizes grandes, também observamos um miss rate elevado. Isso ocorre porque blocos muito grandes podem não caber na cache, levando a substituições sucessivas e perda de eficiência.

Portanto, o balanceamento do block size é crucial. Deve-se evitar block sizes pequenos, que não aproveitam a localidade espacial, e block sizes grandes, que não cabem na cache. O ideal é encontrar um tamanho de bloco que maximize a reutilização dos dados enquanto minimiza as substituições, garantindo que o bloco caiba no cache e otimiza a performance do sistema.